UNIVERSIDADE ESTADUAL DE MARINGÁ
CENTRO DE TECNOLOGIA
DEPARTAMENTO DE INFORMÁTICA

Disciplina: 12035 - Sistemas Operacionais
Professor Dr. Alisson Renan Svaigen

Trabalho Prático

Implementação de Operações de Gestão de Memória para um SO em um Ambiente Computacional Simulado

---

Especificação Geral

O presente trabalho propõe a formulação, modelagem e implementação de operações de gestão de memória para um SO em um ambiente computacional simulado. O ambiente a ser simulado deve considerar a técnica de tradução de endereços **por partições** para virtualização de memória. O SO deve gerenciar solicitações de **alocação** e **liberação de memória,** tal que a ordem de grandeza de armazenamento é dada por *unidades de armazenamento* (UA).

Em termos práticos, considere que cada UA é dado por um índice em uma posição de uma estrutura de vetor (ou lista) na linguagem de programação que você implementará o trabalho!

Devem ser implementadas **quatro estratégias** distintas para alocação/liberação:

- **First-fit (primeiro encaixe)**: escolhe a primeira área livre que satisfaça o pedido de alocação, a partir do endereço de memória menos significativo;
- **Best-fit (melhor encaixe):** escolhe a menor área possível que possa receber a alocação;
- **Worst-fit (pior encaixe):** escolhe sempre a maior área livre possível.
- **Buddy (por pares):** realiza a alocação e liberação por pares considerando blocos de tamanho 2^n, tal que o valor mínimo para *n* é 1 e o valor máximo para *n* é 12, totalizando blocos mínimos de 2 UA e blocos máximos de 4096 UA.

É necessário implementar na simulação uma série de componentes do SO e de hardware para dar suporte às operações. De modo geral, as estruturas a serem implementadas são (mas não necessariamente apenas estas):

- **Memória principal**: representação da memória RAM, cujo tamanho físico é sempre 4096 UA.

- **Lista de processos**: representação que o SO mantém dos processos que estão ativos no sistema;
- **Unidade de Gerenciamento de Memória (MMU)**: representação do componente da MMU para realização das traduções de endereço. A implementação lógica de operação da MMU deve seguir o que foi apresentado em sala de aula no que tange à tradução de endereços de memória virtual por partições;
- **Tabela de partição:** tabela única utilizada pelo SO que mantém o registro das partições já alocadas, armazenando os valores de limite de endereço do processo e início do endereçamento físico na memória física.

Linguagens de Programação permitidas para implementação

Este trabalho pode ser implementado nas seguintes linguagens de programação: Python, C, C++ ou Java. Não serão aceitas implementações em quaisquer outras linguagens além das quatro listadas.

Arquivo de entrada

O funcionamento geral dos componentes implementados, bem como do fluxo geral da simulação, são dependentes de um arquivo de entrada. O arquivo de entrada possui as seguintes informações:

- A primeira linha do arquivo indica a quantidade de processos que estão relacionados à simulação;
- A segunda linha contém os PIDs de processos que podem realizar operações de alocação e liberação de memória. Cada PID é separado por ";"
    - Um PID é sempre dado por uma *string* composta por 3 caracteres.
        - Exemplos: `abc;cde;p01;p02;2px`
- As próximas *n* linhas são constituídas por requisições de:
    - **Alocação de memória**: seguem o formato `aloca [PID][UA]`, representando o PID e a quantidade de memória para ser alocada, respectivamente;
    - **Liberação de memória**: seguem o formato `libera [PID]`, indicando a liberação da região de memória de um determinado processo;
    - **Acesso à um endereço específico de memória**: seguem o formato `acessa [PID][end_log]`, representando o PID e o endereço lógico a ser acessado, respectivamente.

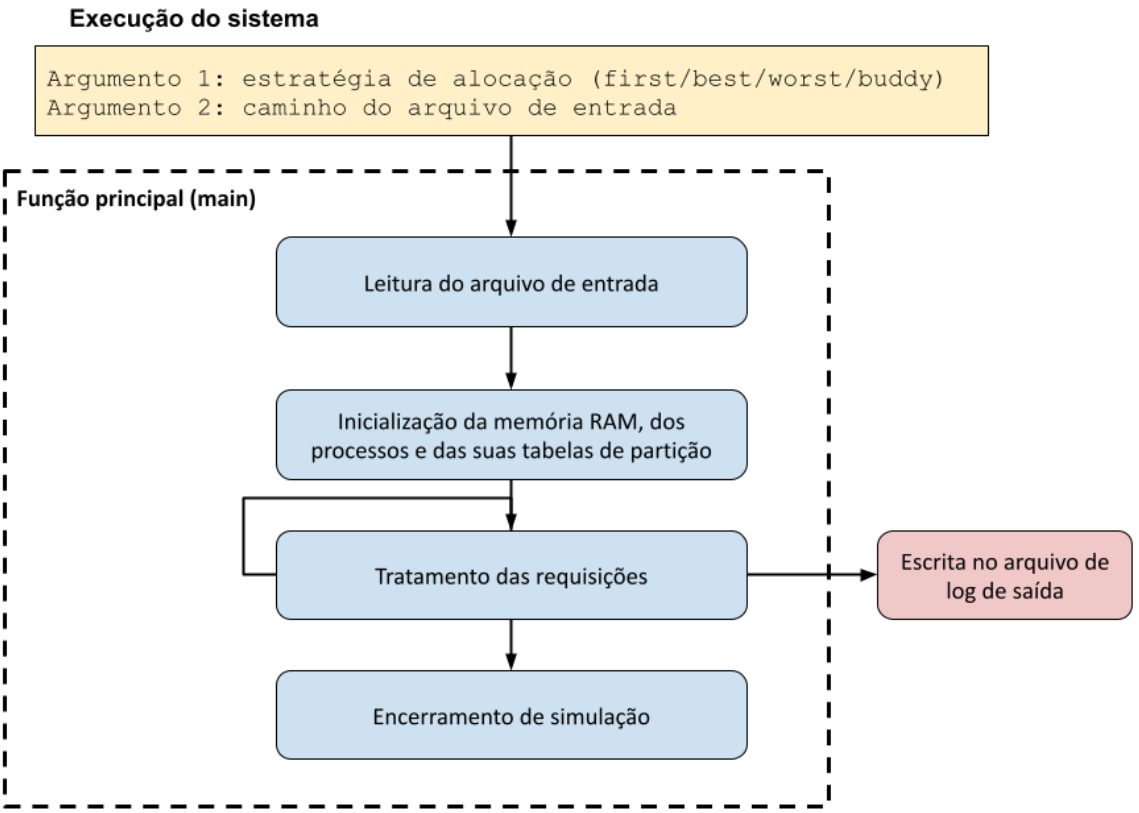
A tabela a seguir mostra um exemplo de entrada possível.

```
4
p00;p01;p02;p03
aloca p03 10
aloca p00 512
aloca p01 24
acessa p03 0
acessa p01 0
acessa p03 9
libera p03
aloca p02 8
```

Nesse exemplo, 4 processos irão, em algum momento, realizar requisições de memória. Uma série de requisições são solicitadas: as três primeiras são de alocação; após, temos três solicitações de acesso; na sequência, uma de liberação e, por fim, uma nova solicitação de alocação.

Fluxo de operação

A figura a seguir apresenta o fluxo de operação esperado do ambiente de simulação. Cada etapa será detalhada na sequência.



- **Execução do sistema de simulação**: o sistema deve ser executado considerando as diretivas para execução da linguagem de programação de implementação. Devem ser considerados dois argumentos: o descritor da estratégia de alocação (first, best, worst ou buddy); e o caminho relativo do arquivo de entrada.
- **Leitura do arquivo de entrada**: a primeira atividade operacional do ambiente de simulação deve ser a leitura do arquivo de entrada, identificando e segmentando as diretivas relacionadas aos processos existentes e ao tratamento de requisições.
- **Inicializações**: de maneira concomitante à leitura, deve ser realizada a inicialização dos principais componentes de simulação (tanto os apresentados na seção anterior quanto os que você julgar necessário criar).
- **Tratamento das requisições**: a principal etapa do fluxo de operação do ambiente simulado consiste no tratamento das requisições realizadas pelos processos. Desse modo, para cada requisição lida do arquivo de entrada, o sistema deve realizar as operações relacionadas a cada requisição. Cada tipo de requisição tem potencial para causar um impacto no estado corrente da memória RAM, na tabela de partições e também na operação da MMU. Portanto, cada tipo de requisição gera uma linha correspondente em um arquivo de log, informando as modificações ocasionadas. O arquivo de saída de log é descrito a seguir.

Escrita no arquivo de log de saída

Para cada simulação realizada, o seu código deve gerar um arquivo de log de saída, cujo nome de arquivo deve seguir o seguinte padrão:

```
log_[nome do arquivo de entrada]_[estratégia].txt
```

Por exemplo, se a simulação está relacionada a um arquivo de entrada de nome “entrada01.txt” e a estratégia de alocação Buddy, o nome do arquivo de log de saída deve ser:

```
log_entrada01_buddy.txt
```

A seguir tem-se uma discussão sobre cada uma das requisições que podem ser solicitadas pelos processos, e como as informações devem ser gravadas em log:

- **Requisição de alocação**: para cada alocação solicitada, o sistema deve tentar alocar a quantidade de espaço solicitada pelo processo seguindo a estratégia de alocação informada pelo 1º argumento de execução. A alocação impacta diretamente na tabela de partição e também no estado corrente da memória RAM.

- Caso **exista** espaço disponível para alocação, deve ser gravado no log uma linha com o seguinte formato:
  - `alocacao [PID] [ini] [fim]`
    - Na qual `[PID]` é o PID do processo solicitante, `[ini]` é o primeiro endereço físico e `[fim]` é o último endereço físico relacionado à alocação.
- Caso **não exista** espaço disponível para alocação, deve ser gravado no log uma linha com o seguinte formato:
  - `alocacao [PID] erro!`
    - Nesse caso, o ambiente de simulação **deve ser encerrado**, invalidando todas as solicitações subsequentes.

- **Requisição de liberação**: para cada liberação solicitada, o sistema deve tornar livre o espaço de alocação utilizado pelo processo solicitante. A liberação impacta diretamente na tabela de partição e também no estado corrente da memória RAM. Em um pedido de liberação de memória, deve ser gravada uma linha no arquivo de log com o seguinte formato:
  - `liberacao [PID] [ini] [fim]`
    - Na qual `[PID]` é o PID do processo solicitante, `[ini]` é o primeiro endereço físico e `[fim]` é o último endereço físico relacionado à liberação de memória.
- **Requisição de acesso à um endereço lógico:** para cada acesso solicitado, o sistema deve ativar a MMU para realização da tradução do endereço. Esse processo não tem impacto direto no estado das estruturas implementadas, uma vez que consiste na execução lógica do código da MMU. No entanto, é possível obter dois resultados possíveis:
  - Caso o endereço lógico solicitado **seja válido**, deve ser gravado no arquivo de log uma linha com o seguinte formato:
    - `acesso [PID] [end_logico] [end_fisico]`
      - Na qual `[PID]` é o PID do processo solicitante, `[end_logico]` é o endereço lógico solicitado e `[end_fisico]` é o endereço físico resultante do processo de tradução da MMU.
  - Caso o endereço lógico solicitado **seja inválido**, deve ser gravado no arquivo de log uma linha com o seguinte formato:
    - `acesso [PID] [end_logico] violacao`
      - Mesmo com a violação detectada, o sistema deve continuar com o fluxo de tratamento das requisições

Para facilitar as tratativas de erro no código a ser implementado, considere as características a seguir como sendo sempre verdade nos arquivos de entrada:

- Um processo solicitará apenas uma vez alocação de memória;
- Um processo solicitará apenas uma vez liberação de memória, ocorrendo sempre numa linha posterior a requisição de alocação;
- Um processo pode solicitar *n* acessos a um endereço de memória, na qual todas as solicitações desse tipo ocorrem após a solicitação de alocação e antes da solicitação de liberação de memória.

Considerando o arquivo de entrada apresentado nesta especificação, o arquivo de log correspondente, considerando como argumento de entrada a estratégia *first-fit*, seria o seguinte:

```
alocacao p3 0 9
alocacao p0 10 521
alocacao p1 522 545
acesso p3 0 0
acesso p1 0 522
acesso p3 9 9
liberacao p3 0 9
alocacao p2 0 7
```

Casos omissos

Quaisquer casos omissos relacionados ao desenvolvimento deste trabalho que não foram esclarecidos nesta descrição deverão ser arguidas diretamente com o professor.

Normas para desenvolvimento e entrega do Trabalho Prático

O trabalho prático deve ser desenvolvido **por uma equipe de, no máximo, três alunos matriculados na disciplina**.

A submissão do trabalho prático será feita exclusivamente via Classroom, na página correspondente à disciplina. **Não serão aceitos trabalhos entregues após o prazo final de cada etapa e também não serão aceitos trabalhos submetidos por outros locais (email, whatsapp, etc.)**.

Os seguintes documentos devem ser entregues:

- Código fonte do ambiente de simulação desenvolvido;

- Arquivo no estilo “README” apresentando qual a versão da linguagem de programação utilizada, como compilar (se for o caso) e executar o ambiente de simulação. Caso seja necessário utilizar alguma biblioteca adicional, identificar nesse arquivo.

Prazo final: 01/07/2026 às 23:59.

Observação: O tempo estipulado para realização deste trabalho é menor do que o prazo final determinado. Mais especificamente, este trabalho possui estimativa de 1 mês letivo para ser completamente desenvolvido. Dessa maneira, alunos atendidos pelo PROPAE já estão contemplados com 50% a mais do tempo necessário para desenvolvimento do trabalho.

Apresentações do Trabalho Prático

- As apresentações irão ocorrer nos dias 03 e 08 de julho de 2026, no horário de aula.
- O cronograma de apresentação será definido unilateralmente pelo professor, sendo divulgado na véspera do primeiro dia de apresentações.
- A apresentação consistirá na demonstração de execução do ambiente de simulação a partir de arquivos de entrada fornecidos pelo professor. Durante a demonstração, o professor se reserva ao direito de:
  - Solicitar que a equipe proceda com a execução de diferentes arquivos;
  - Realizar perguntas para a equipe, ou individualmente, relacionadas à implementação do código;
  - Solicitar a realização de “testes de mesa” a partir do arquivo de entrada, de maneira manual;
  - Realizar perguntas para a equipe, ou individualmente, relacionadas à fundamentação teórica vista em sala de aula, desde que esteja alinhada ao desenvolvimento do trabalho prático.

Considerações Finais

- A apresentação do trabalho é **mandatória** para **toda a equipe.**
- A performance individual e em equipe na apresentação possui **caráter de pontuação eliminatório**, ou seja: caso o professor avalie que o indivíduo, ou a equipe, não demonstram conhecimento sobre o desenvolvimento do trabalho, toda a nota pode ser zerada, mesmo que o trabalho tenha sido implementado e entregue.
- É por meio da apresentação que o professor da disciplina se **certificará de** que todos os membros da equipe contribuíram para a realização do trabalho prático!

- **Se for utilizar ferramentas de IA, utilize com responsabilidade, e saiba aquilo que você está fazendo! Não vá para o Exame Final da disciplina sem necessidade 🙂**